Unité 3 — Cahier de laboratoire (étudiant)

Livrable C — Heap & erreurs temporelles · UE « Software Security »

Code UE 7532 14 U32 D1 · Document étudiant · 11 octobre 2026

Cadre. Laboratoire en environnement isolé (livrable E).

Lab 3.1 — Use-after-free EXERCICE GUIDÉ

1. Objectifs

- Écrire, déclencher et observer un UAF.
- Lire un rapport ASan (libéré ici / alloué ici / utilisé ici).

2. Prérequis

- Cours U3 §4.

3. Environnement

Image de référence (livrable E) : clang + ASan, Valgrind.

4. Fichiers sources

UAF.C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct { void (*action)(void); char nom[16]; } Session;
static void deconnexion(void){ printf("deconnexion executee\n"); }
int main(void){
    Session *s = malloc(sizeof *s);
    s->action = deconnexion;
    strcpy(s->nom, "alice");
    free(s);                /* la memoire est rendue a l'allocateur */
    /* ... plus loin, un pointeur pendouillant est reutilise ... */
    s->action();            /* BUG : use-after-free -> appel indirect */
    return 0;
}
```

5. Commandes

SHELL

```
clang -g -fsanitize=address uaf.c -o uaf_asan
./uaf_asan
```

6. Manipulations

a. Compilez avec ASan et exécutez.

b. Repérez les trois piles d'appels du rapport : *utilisé ici*, *libéré ici*, *alloué ici*.

c. Expliquez pourquoi l'appel via `s->action()` est dangereux après `free`.

7. Résultats attendus

ASan signale `heap-use-after-free` avec les trois localisations (alloc, free, usage).

8. Explications

Après `free`, le slot peut être réalloué ; déréférencer le pointeur pendouillant lit/exécute des octets éventuellement contrôlés par un attaquant.

9. Questions

1. Quelles sont les 3 lignes (alloc/free/usage) ?
2. Quelle correction d'une ligne évite le bug ?
3. Pourquoi l'UAF est-il exploitable (rappel §4) ?

11. Vulnérabilité & remédiation

CWE-416 (use-after-free). Remédiation : `free(s); s=NULL;` et ne plus déréférencer ; propriété claire du pointeur.

12. Durée

45 min.

10. Corrigé : livrable D, Lab 3.1.

Lab 3.2 — Double-free & débordement de tas EXERCICE GUIDÉ

1. Objectifs

- Observer un double-free et un heap overflow sous ASan.

2. Prérequis

- Lab 3.1.

3. Environnement

Image de référence (livrable E) : clang + ASan, Valgrind.

4. Fichiers sources

DOUBLE_FREE.C

```
#include <stdlib.h>
#include <string.h>
int main(void){
    char *p = malloc(32);
    strcpy(p, "donnee");
    free(p);
    /* ... chemin d'erreur qui libere a nouveau ... */
    free(p);                /* BUG : double-free */
    return 0;
}
```

HEAP_OF.C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char **argv){
    char *buf = malloc(16);
    const char *src = (argc>1)? argv[1] : "chaine beaucoup trop longue pour 16";
    strcpy(buf, src);        /* BUG : heap-buffer-overflow si src>15 */
    printf("%s\n", buf);
    free(buf);
    return 0;
}
```

5. Commandes

SHELL

```
clang -g -fsanitize=address double_free.c -o df && ./df
clang -g -fsanitize=address heap_of.c -o hof && ./hof "$(python3 -c 'print("A"*40)')"
```

6. Manipulations

a. Compilez et exécutez chaque programme sous ASan.

b. Pour le double-free, repérez « attempting double-free » et le « freed by thread here ».

c. Pour le heap overflow, notez la taille de l'écriture vs la taille allouée.

7. Résultats attendus

ASan signale `double-free` pour le premier, `heap-buffer-overflow` (WRITE) pour le second.

8. Explications

Le double-free corrompt les listes de l'allocateur ; le heap overflow écrit au-delà de l'objet alloué (ici dans `strcpy`).

9. Questions

1. Pourquoi un double-free est-il dangereux au-delà d'un simple plantage ?
2. Quelle est la taille allouée et la taille écrite dans le cas du heap overflow ?
3. Quelles remédiations ?

11. Vulnérabilité & remédiation

CWE-415 (double-free), CWE-122 (heap overflow). Remédiation : mettre à NULL après free ; copie bornée (`strncpy`/`snprintf`) et allocation de taille suffisante.

12. Durée

45 min.

10. Corrigé : livrable D, Lab 3.2.

Lab 3.3 — Lecture non initialisée (Valgrind) EXERCICE AUTONOME

1. Objectifs

- Détecter une lecture de mémoire non initialisée (qu'ASan ne voit pas).
- Comparer ASan et Valgrind.

2. Prérequis

- Labs 3.1–3.2.

3. Environnement

Image de référence (livrable E) : clang + ASan, Valgrind.

4. Fichiers sources

UNINIT.C

```
#include <stdio.h>
#include <stdlib.h>
int main(void){
    int *t = malloc(4*sizeof(int));
    t[0]=1; t[1]=2;          /* t[2], t[3] non initialises */
    int s=0;
    for(int i=0;i<4;i++) s += t[i];   /* lecture de valeurs non initialisees */
    if (s > 1000) printf("grand\n"); else printf("petit (s=%d)\n", s);
    free(t);
    return 0;
}
```

5. Commandes

SHELL

```
gcc -g -O0 uninit.c -o uninit
valgrind -q --tool=memcheck ./uninit
```

6. Manipulations

a. Compilez normalement et lancez sous Valgrind.

b. Repérez « Conditional jump or move depends on uninitialised value(s) » et la ligne source.

c. Vérifiez qu'un build ASan seul ne signale rien ici ; concluez sur la complémentarité des outils.

7. Résultats attendus

Valgrind signale une dépendance à une valeur non initialisée (ligne de la lecture/branchement).

8. Explications

ASan instrumente surtout les erreurs d'adressage ; les lectures non initialisées relèvent de MemorySanitizer ou de Valgrind memcheck.

9. Questions

1. Quelle ligne Valgrind pointe-t-il ?
2. Pourquoi ASan ne détecte-t-il pas ce bug ?
3. Quelle correction ?

11. Vulnérabilité & remédiation

CWE-457 (use of uninitialized variable). Remédiation : initialiser à l'allocation (`calloc`, `memset`, valeur par défaut).

12. Durée

45 min.

10. Corrigé : livrable D, Lab 3.3.

Programmes conçus pour le cours ; rapports ASan/Valgrind produits et vérifiés.